BioTime Connect Pro: Testing and Installation Guide

For module `biotime_connect_pro` (Odoo 19). Written for the developer who will test, install and publish it.

> **Status:** the module has passed syntax and manifest checks only. It has not yet run on a real Odoo 19 or BioTime server. Treat the first install as a staging test.

1. Prepare a test environment

Use Docker so you can destroy and rebuild it freely. Create a folder, put the unzipped `biotime_connect_pro` folder inside `./addons/`, and add this `docker-compose.yml`:

```
services:
  db:
    image: postgres:16
    environment:
      POSTGRES_USER: odoo
      POSTGRES_PASSWORD: odoo
      POSTGRES_DB: postgres
  odoo:
    image: odoo:19.0
    depends_on: [db]
    ports: ["8069:8069"]
    environment:
      HOST: db
      USER: odoo
      PASSWORD: odoo
    volumes:
      - ./addons:/mnt/extra-addons
    extra_hosts:
      - "host.docker.internal:host-gateway"
```

Start it with `docker compose up -d`, open http://localhost:8069 and create a database named `test` **with demo data**. Check Docker Hub if the `odoo:19.0` tag name differs.

2. Install the module

1. In Odoo, turn on Developer Mode (Settings, then Activate the developer mode).
2. Open Apps, click **Update Apps List**, search for "BioTime" and click Install.
3. Or from the command line (also the fastest way to re-test after code changes):

```
# first install
docker compose exec odoo odoo -d test -i biotime_connect_pro --stop-after-
init
# after editing code
docker compose exec odoo odoo -d test -u biotime_connect_pro --stop-after-
init
docker compose restart odoo
```

If install fails, read the traceback: `docker compose logs odoo --tail 100`. The most likely trouble spots are Odoo 19 view or group syntax. Send me the traceback and I will fix it.

3. Test without a real device (mock BioTime server)

Save this as `mock_biotime.py` and run `python3 mock_biotime.py` on your host. It imitates the BioTime REST API with 9 punches covering the normal and the awkward cases.

```
import json
from http.server import BaseHTTPRequestHandler, HTTPServer
from urllib.parse import urlparse, parse_qs

T = [{"id": 1, "sn": "MOCK001", "alias": "Main Door", "ip_address":
"192.168.1.201",
      "fw_ver": "Ver 6.60", "state": 1, "user_count": 4, "fp_count": 4,
      "face_count": 0, "palm_count": 0, "last_activity": "2026-11-03
09:00:00"}]

def p(i, code, ts, st, first="Emp", last=""):
    return {"id": i, "emp_code": code, "first_name": first, "last_name":
last,
            "punch_time": ts, "punch_state": st,
            "punch_state_display": "Check In" if st == "0" else "Check Out",
            "verify_type_display": "Fingerprint", "terminal_sn": "MOCK001",
            "terminal_alias": "Main Door", "area_alias": "HQ"}
```

```
PUNCHES = [
    p(1, "100", "2026-11-03 09:12:00", "0", "Ali", "Khan"),   # late
    p(2, "100", "2026-11-03 18:05:00", "1", "Ali", "Khan"),
    p(3, "101", "2026-11-03 08:58:00", "0", "Sara", "Ahmed"), # on time
    p(4, "101", "2026-11-03 17:30:00", "1", "Sara", "Ahmed"),
    p(5, "102", "2026-11-03 22:00:00", "0", "Omar", "Night"), # night shift
    p(6, "102", "2026-11-04 06:00:00", "1", "Omar", "Night"),
    p(7, "103", "2026-11-03 09:00:00", "0", "Zed", "Double"),
    p(8, "103", "2026-11-03 09:00:30", "0", "Zed", "Double"), # double punch
    p(9, "999", "2026-11-03 10:00:00", "0", "Unknown", "Person"),
]
STORE = {"employees": [], "departments": [], "areas": []}
PATHS = {"/personnel/api/employees/": "employees",
         "/personnel/api/departments/": "departments",
         "/personnel/api/areas/": "areas"}
KEY = {"employees": "emp_code", "departments": "dept_code", "areas":
"area_code"}

class H(BaseHTTPRequestHandler):
    def _send(self, obj, code=200):
        body = json.dumps(obj).encode()
        self.send_response(code)
        self.send_header("Content-Type", "application/json")
        self.send_header("Content-Length", str(len(body)))
        self.end_headers()
        self.wfile.write(body)

    def _body(self):
        n = int(self.headers.get("Content-Length", 0))
        return json.loads(self.rfile.read(n) or b"{}")

    def do_GET(self):
        u = urlparse(self.path); q = {k: v[0] for k, v in
parse_qs(u.query).items()}
        if u.path == "/iclock/api/terminals/":
            return self._send({"count": len(T), "next": None, "data": T})
        if u.path == "/iclock/api/transactions/":
            rows = [r for r in PUNCHES if r["punch_time"] >=
q.get("start_time", "")]
            if "emp_code" in q: rows = [r for r in rows if r["emp_code"] ==
q["emp_code"]]
            if "terminal_sn" in q: rows = [r for r in rows if
```

```
r["terminal_sn"] == q["terminal_sn"]]
            return self._send({"count": len(rows), "next": None, "data":
rows})
        if u.path in PATHS:
            name = PATHS[u.path]; rows = STORE[name]
            k = KEY[name]
            if k in q: rows = [r for r in rows if str(r.get(k)) == q[k]]
            return self._send({"count": len(rows), "next": None, "data":
rows})
        self._send({"detail": "not found"}, 404)

    def do_POST(self):
        u = urlparse(self.path)
        if u.path in ("/jwt-api-token-auth/", "/api-token-auth/"):
            return self._send({"token": "mock-token"})
        if u.path in PATHS:
            rows = STORE[PATHS[u.path]]
            obj = dict(self._body(), id=len(rows) + 10); rows.append(obj)
            return self._send(obj, 201)
        self._send({"detail": "not found"}, 404)

    def do_PUT(self):
        u = urlparse(self.path); base = u.path.rsplit("/", 2)[0] + "/"
        if base in PATHS:
            self._send(self._body())
        else:
            self._send({"detail": "not found"}, 404)

HTTPServer(("0.0.0.0", 8000), H).serve_forever()
```

In Odoo create a server with: host `host.docker.internal`, port `8000`, protocol HTTP, auth type **JWT**, any username and password, device timezone as you prefer.

Prepare the Odoo side before pulling:

- Create employees Ali Khan (code `100`), Sara Ahmed (`101`), Omar Night (`102`) and Zed Double (`103`). Put the code in the employee's **BioTime** tab.
- Give Ali and Sara a 09:00 to 18:00 working schedule (Mon to Fri). Give Omar a 22:00 to 06:00 schedule. Without a matching schedule, the status shows "No Schedule".
- Do not create employee `999`; it tests the error path.

4. Test checklist

Run these in order. Expected results assume the mock data above and server defaults (5 min late grace, 1 min double-punch gap).

#	Test	How	Expected result
1	Connection	Server, Test Connection	Green notice: "1 device(s) found"
2	Wrong password / host	Change host, test again	Clear error message, no crash
3	Devices	Sync Devices	Device MOCK001 online, 4 users
4	Pull	Pull Transactions wizard (from 2026-11-03 00:00), transfer ON	9 punches read, 9 new
5	Results	Transactions list	7 Processed, 1 Skipped (id 8), 1 Error (id 9, no employee)
6	Late rule	BioTime, Attendance	Ali: Late In, 7 min (12 minus 5 grace). Sara: On Time
7	Night shift	Same list	Omar has one record, 22:00 to 06:00, not two
8	Duplicate-safe	Pull the same period again	0 new punches
9	Error recovery	Create employee 999, then Retry on punch 9	Punch becomes Processed
10	Net hours / overtime	Pivot or list	Ali: worked ~8.9 h, minus 1 h break, so net ~7.9 h
11	Schedule change	Change Ali's working hours to 10:00 to 19:00	Ali's late minutes recompute automatically
12	Pairing mode	Switch to Alternating, reset punches, reprocess	Same attendance result
13	Job retry	Stop the mock, click Queue Pull, wait	Job goes Pending, retries after 1, 5, 10, 30 min, then Failed; Requeue works

#	Test	How	Expected result
14	Push	Push Master Data	Mock `STORE` holds your departments, areas, employees; second push updates instead of duplicating
15	Employee pull	Enable auto-create, Pull Employees	Employees appear (mock returns none until you POST some)
16	Permissions	Log in as BioTime User, Supervisor, Manager	User: read only. Supervisor: process and retry punches. Manager: servers and passwords
17	Multi-company	Create a second company and server	Users see only their company's data
18	Crons	Settings, Technical, Scheduled Actions	Three BioTime crons exist; run each manually once
19	Uninstall / upgrade	`-u` then uninstall from Apps	No errors

If test 6 shows "No Schedule" for everyone, the schedule lookup in `hr_attendance.py` (`_bt_shift`) needs adjusting for Odoo 19's calendar API. Send me the result.

5. Test with the real BioTime

After the mock passes:

1. Install BioTime 8.5 or 9 (ZKTeco offers trial installs), or use a customer's server.
2. Try auth types in this order: **JWT** (BioTime 8.x and Cloud), then **Token** (9.x), then Basic. A 401 or 404 on login means the wrong type.
3. Open the BioTime web UI and compare terminal list, employee codes and a few punches against what Odoo imported.
4. Check **timezone**: punch at a known time on a device and confirm the Odoo check-in matches after conversion.

5. Check **punch states**: some devices send values other than 0 and 1 (break, overtime). Unmapped states are skipped on purpose. Adjust the IN/OUT mapping or use Alternating mode.
6. Do a large pull (for example a month) in background mode to verify paging and speed.
7. Run 24 hours with automatic sync and compare headcounts.

6. Automated tests (recommended before selling)

There are no unit tests in the module yet. Add a `tests/` folder with `TransactionCase` tests for the processing engine (pairing, double punch, stale open attendance, late minutes). Run them with:

```
docker compose exec odoo odoo -d test_ci -i biotime_connect_pro --test-enable
--stop-after-init
```

I can write these tests for you as the next step.

7. Common problems

Symptom	Likely cause and fix
Install error about a view or group	Odoo 19 syntax difference. Read the traceback and fix that line
"Cannot reach host"	In Docker use `host.docker.internal`, not `localhost`. Check port and firewall
Authentication failed	Wrong auth type, or the BioTime API user lacks permission
Punches off by hours	Device Timezone on the server record is wrong
Many punches in Error	Employee code not set, or Mapping Field setting does not match
Cron not firing	Check cron is active and the Odoo worker is running

8. Adding it to Odoo

8.1 Your own Odoo (on-premise or Docker)

1. Copy the `biotime_connect_pro` folder into a directory on the `addons_path` (in `odoo.conf`).
2. Make sure Python packages `requests` and `pytz` are installed (normally already present).
3. Restart Odoo, then Apps, **Update Apps List**, search and install.

8.2 Odoo.sh

Put the folder at the root of your project's Git repository (or in a submodule), push to a branch, and Odoo.sh builds it. Install from Apps as above.

8.3 Odoo Online (SaaS)

Odoo Online generally does not run custom Python modules. Confirm with Odoo's current documentation or support before promising it to customers. Listings on the Apps store state their availability.

8.4 Publishing on the Odoo Apps store (apps.odoo.com)

The steps below come from a third-party guide and my knowledge of the store; confirm details in Odoo's official publishing documentation before you submit.

Before you submit, fix these in the module:

- Replace the placeholder `author` and `website` in `__manifest__.py`.
- Add `static/description/icon.png` (128x128), `banner.png` (about 1280x600) and 3 to 5 screenshots (about 1200x800). Then add `'images': ['static/description/banner.png']` back to the manifest.
- Add a `LICENSE` file (Odoo Proprietary License v1, matching `OPL-1`). Paid apps need OPL-1; open-source licenses cannot charge on the store.
- Replace `index.html` text with screenshots and a short demo video link.
- Keep the version in `__manifest__.py` equal to the folder and branch version (`19.0.x.x.x`).
- Remove any hard-coded company data. Optionally run `pylint-odoo` for quality warnings.

Package and verify:

```
zip -r biotime_connect_pro_19.0.1.0.0.zip biotime_connect_pro -x "*.pyc"
"*/__pycache__/*" "*/.git/*"
unzip -l biotime_connect_pro_19.0.1.0.0.zip | head   # folder name must be at
the top
```

Test the zip on a **fresh** database: install, upgrade (`-u`), then uninstall. All three must be clean, with and without demo data.

Submit:

1. Create or sign in to your Odoo account and open the Apps store developer area (apps.odoo.com, Publish a Module).
2. Fill in module name, category, supported version (19.0), upload the zip, banner, screenshots, description and **price** ($149 launch, $199 regular as recommended earlier).
3. Wait for review. A third-party source quotes 5 to 10 business days; common rejections are a missing license field, hard-coded company data and code quality warnings.
4. For each future Odoo version, maintain a separate branch or package (18.0, 19.0, and so on).

Money and support: the same source says Odoo keeps about 30% of each sale. Verify this in the Apps store terms. Offer a support channel (email or WhatsApp), since buyers on similar listings ask for contact details before buying.

8.5 Selling outside the store

You can also send the zip directly to customers or host a private Git repository. With OPL-1 you remain the licence holder, but without the store you must handle payment, delivery and updates yourself.

9. Recommended order of work

1. Install on Docker and fix any install errors.
2. Run the mock-server checklist (section 4).
3. Test against a real BioTime (section 5).
4. Add unit tests and screenshots.
5. Package, test on a fresh database, and submit.